

Indexers Real-Time Example in C#

In this article, I am going to discuss **Indexers Real-Time Example in C#**. Please read our previous article before proceeding to this article where we discussed [What indexers are and How to Create and Use Indexers in C#](#) with Examples. As we already discussed in our previous article that an indexer is a member of a class that enables an object (i.e. instance) to be indexed like an array.

Indexer Real-Time Example in C#:

Let us understand Indexers with one Real-Time Example in C#. First, create a class file with the name **Employee.cs** and then copy and paste the following code. This is a very simple class having four properties.

```
namespace IndexersDemo
{
    public class Employee
    {
        public int EmployeeId { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public double Salary { get; set; }
    }
}
```

Next, create another class file with the name **Company.cs** and then copy and paste the following code into the class. In below code, first we creating a variable called listEmployees to hold list of Employees. Within the class constructor, we are initializing the variable listEmployees by adding employees to the list. We then created an integer indexer by using the this keyword. The indexer takes the employeeId as a parameter and returns that employee name. Just like properties, the indexers have both get and set accessors. The get accessor is used to return the value whereas the set accessor is used to assign a value. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System.Collections.Generic;
using System.Linq;
namespace IndexersDemo
{
    public class Company
    {
        //Create a variable to hold list of Employees
        private List<Employee> listEmployees = new List<Employee>();
        //Through the constructor initialize the listEmployees variable
        public Company()
        {
            listEmployees.Add(new Employee { EmployeeId = 101, Name = "Pranay",
            "Male", Salary = 1000 });
            listEmployees.Add(new Employee { EmployeeId = 102, Name = "Preety",
            "Female", Salary = 2000 });
            listEmployees.Add(new Employee { EmployeeId = 103, Name = "Anurag",
            "Male", Salary = 5000 });
        }
    }
}
```

```

        listEmployees.Add(new Employee { EmployeeId = 104, Name = "Priyank",
        = "Female", Salary = 4000 });
        listEmployees.Add(new Employee { EmployeeId = 105, Name = "Hina",
        "Female", Salary = 3000 });
        listEmployees.Add(new Employee { EmployeeId = 106, Name = "Sambit",
        "Male", Salary = 6000 });
        listEmployees.Add(new Employee { EmployeeId = 107, Name = "Tarun",
        "Male", Salary = 8000 });
        listEmployees.Add(new Employee { EmployeeId = 108, Name = "Santosh",
        "Male", Salary = 7000 });
        listEmployees.Add(new Employee { EmployeeId = 109, Name = "Trupti",
        "Female", Salary = 5000 });
    }
    // Integer Indexer
    // The indexer takes an employeeId as parameter and returns the employee
    public string this[int employeeId]
    {
        get
        {
            //Using get accessor return the name of the Employee
            return listEmployees.FirstOrDefault(x => x.EmployeeId ==
employeeId).Name;
        }
        set
        {
            //Using set accessor return the name of the Employee
            listEmployees.FirstOrDefault(x => x.EmployeeId == employeeId).
value;
        }
    }
}

```

Next, modify the Main method of the Program class as shown below. In the below code, first we create an instance of the Company class and then access the Employee name using the Integer Indexer of the Company Object by passing the Employee ID. Again, using the Company Indexer, we are setting the Name of the Employee. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace IndexersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of the Company class
            Company company = new Company();
            //Accessing the Name of the Employee using Integer Indexer of Company
            Console.WriteLine("Name of Employee with Id = 101: " + company[101]);
            Console.WriteLine("Name of Employee with Id = 105: " + company[105]);
        }
    }
}

```

```

        Console.WriteLine("Name of Employee with Id = 107: " + company[107]);

        Console.WriteLine();
        Console.WriteLine("Changing the names of employees with Id = 101,105,107");
        //Setting the Name of the Employee using Integer Indexer of Company class
        company[101] = "Employee 101 Name Changed";
        company[105] = "Employee 105 Name Changed";
        company[107] = "Employee 107 Name Changed";
        Console.WriteLine();
        //Accessing the Name of the Employee using Integer Indexer of Company class
        Console.WriteLine("Name of Employee with Id = 101: " + company[101]);
        Console.WriteLine("Name of Employee with Id = 105: " + company[105]);
        Console.WriteLine("Name of Employee with Id = 107: " + company[107]);
        Console.ReadLine();
    }
}

```

Points to remember:

The EmployeeId's 101,105 and 107 are passed into the company object to retrieve the respective employee names. To retrieve the names of the employees, here the "get" accessor of the indexer is used. Similarly To change the names of the employees, here the set accessor of the integral indexer defined on the Company class is used.

```

company[101] = "Employee 101 Name Changed";
company[105] = "Employee 105 Name Changed";
company[107] = "Employee 107 Name Changed";

```

So when you run the above application code, you will get the following output.

```

Name of Employee with Id = 101: Pranaya
Name of Employee with Id = 105: Hina
Name of Employee with Id = 107: Tarun

Changing the names of employees with Id = 101,105,107

Name of Employee with Id = 101: Employee 101 Name Changed
Name of Employee with Id = 105: Employee 105 Name Changed
Name of Employee with Id = 107: Employee 107 Name Changed

```

Notice that because of the employeeId indexer, now we are able to use the company object like an array.

Overloading Indexer in C#

We can also overload the indexers in C#. That is we can define multiple indexers in class. Let us understand this with an example. As of now, we have an integer indexer in the Company class. Now let us create another indexer based on the string parameter of the company class.

```

//String Indexer
//The indexer takes an gender as parameter and returns the total count of employees based on the gender
public string this[string gender]
{
    get
    {
        // Returns the total count of employees whose gender matches
        // with the gender that is passed in.
        return listEmployees.Count(x => x.Gender.ToLower() == gender.ToLower()).ToString();
    }
    set
    {
        // Changes the gender of all employees whose gender matches
        // with the gender that is passed in.
        foreach (Employee employee in listEmployees)
        {
            if (employee.Gender == gender)
            {
                employee.Gender = value;
            }
        }
    }
}
}

```

The important point to keep in mind is that the indexers are overloaded based on the number and type of parameters. The complete code of the Company class is given below.

```

using System.Collections.Generic;
using System.Linq;
namespace IndexersDemo
{
    public class Company
    {
        //Create a variable to hold list of Employees
        private List<Employee> listEmployees = new List<Employee>();
        //Through the constructor initialize the listEmployees variable
        public Company()
        {
            listEmployees.Add(new Employee { EmployeeId = 101, Name = "Pranaya",
            "Male", Salary = 1000 });
            listEmployees.Add(new Employee { EmployeeId = 102, Name = "Preety",
            "Female", Salary = 2000 });
            listEmployees.Add(new Employee { EmployeeId = 103, Name = "Anurag",
            "Male", Salary = 5000 });
            listEmployees.Add(new Employee { EmployeeId = 104, Name = "Priyank",
            "Female", Salary = 4000 });
            listEmployees.Add(new Employee { EmployeeId = 105, Name = "Hina",
            "Female", Salary = 3000 });
            listEmployees.Add(new Employee { EmployeeId = 106, Name = "Sambit",
            "Male", Salary = 6000 });
            listEmployees.Add(new Employee { EmployeeId = 107, Name = "Tarun",
            "Male", Salary = 8000 });
        }
    }
}

```

```

        listEmployees.Add(new Employee { EmployeeId = 108, Name = "Santosh",
"Male", Salary = 7000 });
        listEmployees.Add(new Employee { EmployeeId = 109, Name = "Trupti",
"Female", Salary = 5000 });
    }
    // Integer Indexer
    // The indexer takes an employeeId as parameter and returns the employee
    public string this[int employeeId]
    {
        get
        {
            //Using get accessor return the name of the Employee
            return listEmployees.FirstOrDefault(x => x.EmployeeId ==
employeeId).Name;
        }
        set
        {
            //Using set accessor return the name of the Employee
            listEmployees.FirstOrDefault(x => x.EmployeeId == employeeId).
value;
        }
    }
    //String Indexer
    //The indexer takes an gender as parameter and returns the total count
employees based on the gender
    public string this[string gender]
    {
        get
        {
            // Returns the total count of employees whose gender matches
            // with the gender that is passed in.
            return listEmployees.Count(x => x.Gender.ToLower() ==
gender.ToLower()).ToString();
        }
        set
        {
            // Changes the gender of all employees whose gender matches
            // with the gender that is passed in.
            foreach (Employee employee in listEmployees)
            {
                if (employee.Gender == gender)
                {
                    employee.Gender = value;
                }
            }
        }
    }
}

```

Notice that, now the Company Class has 2 indexers. The first indexer has an integer (employeeId) parameter and the second indexer has got a string (gender) parameter. To test the string indexer, that we have just created, copy and paste the following code in the Main method of the Program class as shown below.

```
using System;
namespace IndexersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create an Instance of the Company Class
            Company company = new Company();
            Console.WriteLine("Before changing the Gender of all the male employees  
Female");
            Console.WriteLine();
            // Get accessor of string indexer is invoked to return the total number of  
male employees
            Console.WriteLine("Total Number Employees with Gender = Male:" + company["Male"]);
            Console.WriteLine();
            Console.WriteLine("Total Number Employees with Gender = Female:" + company["Female"]);
            Console.WriteLine();
            // Set accessor of string indexer is invoked to change the gender of all  
employees to "Female"
            company["Male"] = "Female";
            Console.WriteLine("After changing the Gender of all male employees to  
Female");
            Console.WriteLine();
            Console.WriteLine("Total Employees with Gender = Male:" + company["Male"]);
            Console.WriteLine();
            Console.WriteLine("Total Employees with Gender = Female:" + company["Female"]);
            Console.ReadLine();
        }
    }
}
```

When we run the application, it gives us the below output.

Before changing the Gender of all the male employees to Female

Total Number Employees with Gender = Male:5

Total Number Employees with Gender = Female:4

After changing the Gender of all male employees to Female

Total Employees with Gender = Male:0

Total Employees with Gender = Female:9

In the next article, I am going to discuss [Enums in C# with Examples](#). Here, in this article, I try to explain **Indexers Real-Time Example in C#**. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.